

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: РАЗРАБОТКА ВИЗУАЛИЗАТОРА АЛГОРИТМА
ТОПОЛОГИЧЕСКОЙ СОРТИРОВКИ
НА ЯЗЫКЕ KOTLIN**

Студент гр. 4342

Я.С. Купчинский

Студент гр. 4342

С.Р. Садыков

Студент гр. 4342

М.Ю. Тиссен

Руководитель

Р.П. Шестопапов

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Я.С. Купчинский Группа: 4342

Студент: М.Ю. Тиссен Группа: 4342

Студент: С.Р. Садыков Группа: 4342

Тема практики: Разработка визуализатора алгоритма топологической сортировки на языке Kotlin

Задание на практику:

Командная итеративная разработка визуализатора алгоритма на Kotlin с графическим интерфейсом.

Алгоритм: Кана для топологической сортировки.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент	Я.С. Купчинский
Студент	М.Ю. Тиссен
Студент	С.Р. Садыков
Руководитель	Р.П. Шестопалов

АННОТАЦИЯ

В рамках учебной практики разработан визуализатор алгоритма топологической сортировки Кана. Приложение реализовано на языке Kotlin с использованием веб-технологий: фреймворка KVision, библиотеки vis.js для отрисовки графа и серверной части на Ktor. Пользователь может задавать ориентированный граф вручную или загрузкой из JSON-файла, после чего в пошаговом режиме наблюдать за выполнением алгоритма с автоматическим вычислением входящих степеней, помещением вершин в очередь и обнаружением циклов. Графический интерфейс обеспечивает полное управление мышью. Проведено тестирование корректности алгоритма и интерфейса. В ходе работы освоены язык Kotlin, принципы клиент-серверного взаимодействия, командная итеративная разработка.

SUMMARY

This report describes the development of a visualizer for Kahn's topological sorting algorithm as part of an internship. The application is built in Kotlin using KVision, vis.js for graph rendering, and Ktor for the backend. It allows the user to create a directed graph interactively or by loading a JSON file and then step through the sorting process while observing in-degree calculations, queue operations, and cycle detection. The graphical interface is fully mouse-driven. Algorithm and UI testing confirmed correctness and usability. The project provided practical experience with Kotlin, web client-server architecture, and team-based iterative development.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 ПОСТАНОВКА ЗАДАЧИ.....	7
1.1 Предметная область.....	7
1.2 Задачи практики.....	7
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	8
2.1. Вводное задание: прохождение курса по изучению языка Kotlin.....	8
2.2. Составление спецификации программы и плана разработки.....	8
2.2.1 Исходные требования к программе.....	8
2.2.2 План разработки и распределение ролей в бригаде.....	9
2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом.....	10
2.3.1. Структуры данных.....	10
2.3.2. Основные методы.....	10
2.3.3. Тестирование.....	11
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ.....	16
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	17
17	
ЗАКЛЮЧЕНИЕ.....	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	19
ПРИЛОЖЕНИЕ А.....	20

ВВЕДЕНИЕ

В рамках данной практики перед бригадой была поставлена цель разработать программный продукт, представляющий из себя приложение, позволяющее визуально взаимодействовать с алгоритмом топологической сортировки.

Предметной областью практики является разработка визуализатора алгоритма топологической сортировки Кана на языке Kotlin. Визуализация пошаговой работы алгоритма на произвольном ориентированном графе позволяет наглядно демонстрировать его логику, что делает продукт полезным как для студентов, изучающих алгоритмы, так и для преподавателей в качестве учебного инструмента.

Целью практики было создание интерактивного веб-приложения с графическим интерфейсом, которое позволяет пользователю задавать граф вручную или загружать из файла и затем в пошаговом режиме наблюдать выполнение алгоритма Кана с отображением текущих входящих степеней, состояния очереди и итогового топологического порядка.

Для достижения поставленной цели были сформулированы следующие задачи практики:

- Прохождение вводного онлайн-курса по языку Kotlin, освоение его базового синтаксиса, объектно-ориентированных возможностей и работы с асинхронностью.
- Составление согласованной с руководителем спецификации программы и плана итеративной разработки, а также распределение ролей внутри бригады.
- Изучение фреймворка KVision и библиотеки vis.js как инструментов для построения клиент-серверных веб-приложений с интерактивной визуализацией графов.
- Командная поэтапная реализация приложения в соответствии со спецификацией, включая написание алгоритмического ядра, серверной части, пользовательского интерфейса и проведение тестирования.

- Оформление отчётной документации по результатам выполненной работы.

Успешное выполнение перечисленных задач позволило создать работоспособный программный продукт, подробно описанный в настоящем отчёте.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Данная работа представляет собой реализацию веб-приложения с графическим интерфейсом, которое позволит пользователю провести топологическую сортировку произвольного графа в пошаговом режиме. Данное приложение позволит наглядно изучить работу алгоритма Кана, что может быть полезно студентам или преподавателям в демонстрационных целях.

1.2 Задачи практики

- Вводное задание: Прохождение курса по изучению языка программирования Kotlin.
- Составление спецификации программы и плана разработки.
- Изучение модуля KVision как инструмента веб визуализации.
- Командная итеративная разработка приложения в соответствии со спецификацией и планом.

2 РЕЗУЛЬТАТЫ РАБОТЫ

В результате было создано приложение, полностью выполняющее следующие требования: Приложение имеет графический интерфейс, который позволяет взаимодействовать с ним используя исключительно мышь, приложение даёт пользователю задать граф как при помощи ручного ввода, так и при помощи импорта из файла формата json, работа алгоритма Кана показана пошагово и полностью контролируется пользователем.

2.1. Вводное задание: прохождение курса по изучению языка Kotlin

В ходе изучения онлайн курса были изучены такие темы, как базовые структуры и функции языка, работа со строками, библиотека Math, основы ООП, наследование, enum классы и nullability.

2.2. Составление спецификации программы и плана разработки

В этом разделе приводятся исходная спецификация, план разработки и распределение ролей в бригаде, согласованные с руководителем, а также уточнения требований, принятые после сдачи промежуточных версий программы.

2.2.1 Исходные требования к программе

2.2.1.1 требования к вводу исходных данных

Входные данные могут быть заданы пользователем интерактивно с помощью мыши на холсте или загружены из текстового файла. При загрузке из файла используется простой текстовый формат, где сначала построчно описываются вершины, а затем рёбра. Строка вершины содержит её уникальный текстовый идентификатор и координаты на холсте (X и Y), разделенные пробелом. Строка ребра содержит идентификатор начальной и конечной вершины, также разделенные пробелом. Строки, начинающиеся с символа решётки, считаются комментариями и игнорируются. Поскольку алгоритм работает с ориентированным графом, порядок указания вершин в строке ребра имеет значение: первое значение указывает на вершину, из которой выходит ребро, а второе - на вершину, в которую оно входит.

2.2.1.2 требования к визуализации

Интерфейс разделен на три области: Левая часть - область отображения графа (canvas), где пользователь создаёт вершины и рёбра. На каждой вершине отображается число - текущая входящая степень. Правая часть - панель управления со следующими кнопками: Назад -> - шаг, Вперёд <-, Старт, Стоп. Также представление результата работы алгоритма с пошаговыми действиями. Левая нижняя часть – кнопка загрузки готового графа из файла типа JSON.

2.2.1.3 требования к функционалу приложения

Программа позволяет: создавать и редактировать ориентированный граф, автоматически рассчитывать входящие степени вершин, пошагово выполнять алгоритм Кана с использованием очереди, выявлять циклы в графе, выводить текстовые пояснения на каждом шаге, формировать итоговый список отсортированных вершин. Все функции доступны через графический интерфейс.

2.2.2 План разработки и распределение ролей в бригаде

Этап 1, прототип, плановая дата выполнения 10.7. Реализация структуры данных графа, базовой логики алгоритма Кана и организация ввода-вывода через стандартный поток (stdin/stdout).

Этап 2, альфа, плановая дата выполнения 12.7. Перенос вычислительного ядра на клиент-серверную или веб-архитектуру и создание простейшего интерфейса для взаимодействия

Этап 3, бета, плановая дата выполнения 14.7. Реализация полноценного графического интерфейса пользователя (GUI) согласно утвержденному прототипу спецификации.

Этап 4, бета, плановая дата выполнения 16.7. Проведение приемочного тестирования, оптимизация графического движка и устранение мелких дефектов.

Роли были распределены следующим образом:

Тиссен М.Ю.- реализация базовых классов, реализация функции обработки файлов, организация передачи файла в память программы из

пользовательского интерфейса, механизм обработки исключений, тестирование и промежуточные задачи.

Купчинский Я.С. – написание алгоритма Кана, организация ввода-вывода из стандартного потока, рефакторинг кода и организация работы git репозитория, тестирование, создание концепта дизайна приложения.

С.Р. Садыков – Создание графического интерфейса, написание логики взаимодействия интерфейса и backend-части приложения. Разработка механизмов взаимодействия пользователя с приложением, механизма навигации, интеграция цветовой гаммы.

2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом

Разработка проходила в более быстром темпе, чем было запланировано. Прототип был разработан в течении суток после утверждения спецификации, а именно 08.07. Этапы 1 и 2 были выполнены за двое суток суммарно, так как модуль KVision позволил использовать наработки группы из предыдущих проектов, но потребовалась серьёзная работа над организацией ввода-вывода. Это позволило гораздо быстрее перейти к итоговому виду и функционалу приложения. Этап четыре потребовал одни сутки выполнения, так как тестирование и доработка проводилась в ходе выполнения работы.

2.3.1. Структуры данных

Класс Edge- описывает ребро графа и имеет два поля VertexId обозначающие стартовую и конечные вершины.

Класс Graph- содержит массив узлов и список рёбер графа.

Класс Vertex- имеет уникальный идентификатор вершины, её расположение и количество входящих рёбер.

Класс GraphJsonParser- класс, отвечающий за обработку входных данных из json файла. buildVertices, которая используется для корректной отрисовки узлов. Для этого используется вспомогательная сетка не отображаемая пользователю.

2.3.2. Основные методы

Методы класса `GraphJsonParser`

Метод `parse`, которая получает на вход название файла и возвращает структуру `Graph`.

Метод `buildVertices`, который используется для корректной отрисовки узлов. Для этого используется вспомогательная сетка, не отображаемая пользователю.

`Graph.calculateInDegrees()` – Рассчитывает входящие степени для всех вершин графа.

`Graph.buildAdjacencyList()` - Строит список смежности, чтобы быстро находить все исходящие соседние вершины

`Graph.initializeStack()` - Собирает все вершины без входящих рёбер.

`Graph.topologicalSortKahn(mode: SortMode)` - Топологическая сортировка Кана. Возвращает топологический порядок, если граф ациклический. Возвращает `null`, если из-за цикла удалось обработать не все вершины.

Так же вспомогательные функции, такие как

`Fun waitForUser(message: String)` - Ждёт ввод пользователя в пошаговом режиме.

`Fun printState(step: Int, result: List<VertexId>, stack: ArrayDeque<VertexId>, vertices: Map<VertexId, Vertex>)` - Выводит текущее состояние алгоритма для отладки и демонстрации: частичный результат, текущий стек и `inDegree` каждой вершины.

`VisDataSet` — реактивный контейнер данных для вершин и ребер.

`VisNetwork` — сам класс интерактивной отрисовки графа, который принимает HTML-контейнер, опции физики/стилей и вешает слушатели событий.

2.3.3. Тестирование

Подраздел тестирования следует структурировать:

2.3.3.1. Тестирование графического интерфейса

Тестирование интерфейса производилось на локальном запуске веб приложения на порте localhost. Изначально была протестирована возможность добавления узла. Тесты были пройдены успешно. На картинке показан один из тестов. (рис. 1).

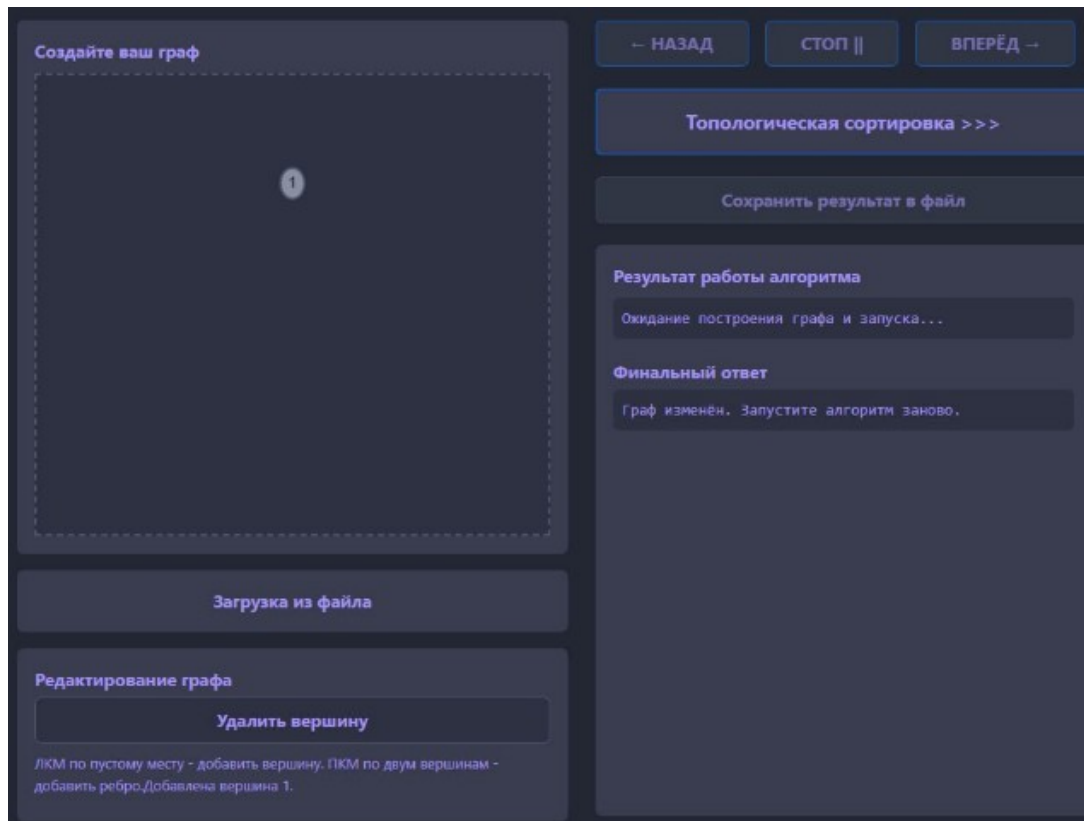


Рисунок 1 – Пример теста добавления узла

Далее была проверена возможность соединения двух узлов направленным ребром. Тесты были пройдены успешно. На картинке показан один из тестов. (рис. 2).

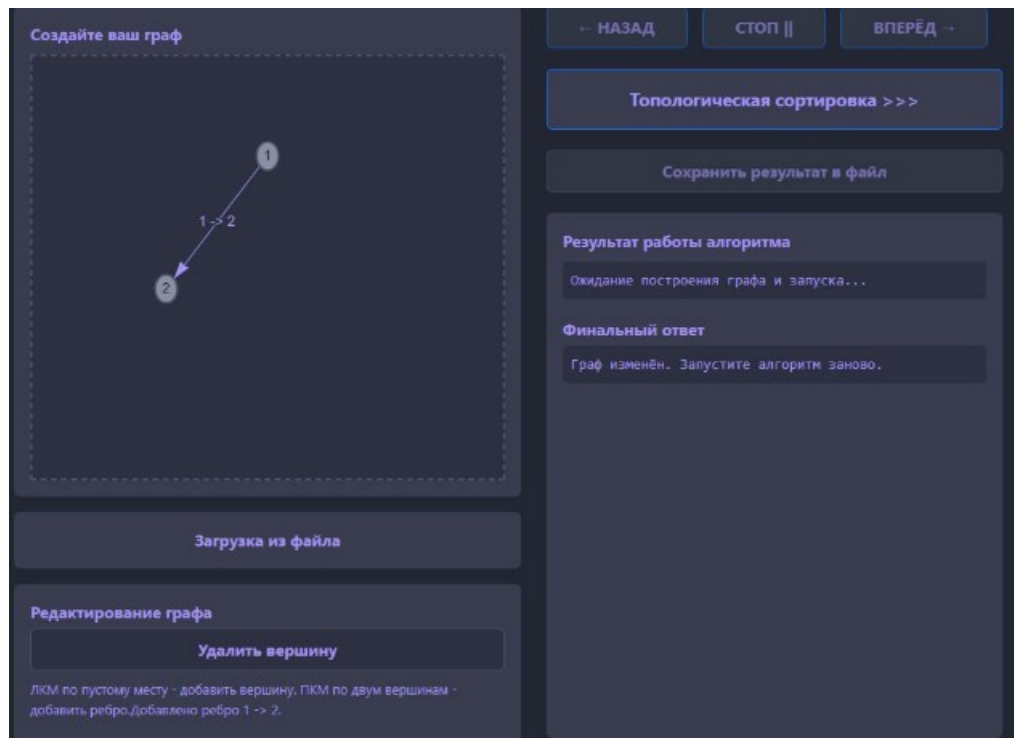


Рисунок 2 – Пример теста добавления ребра

Далее была проверена работа алгоритма в тех же сценариях, как и в алгоритмическом тестировании. Тесты дали тот же ожидаемый результат. Далее приведён пример ожидаемо корректного результата работы программы (рис. 3).

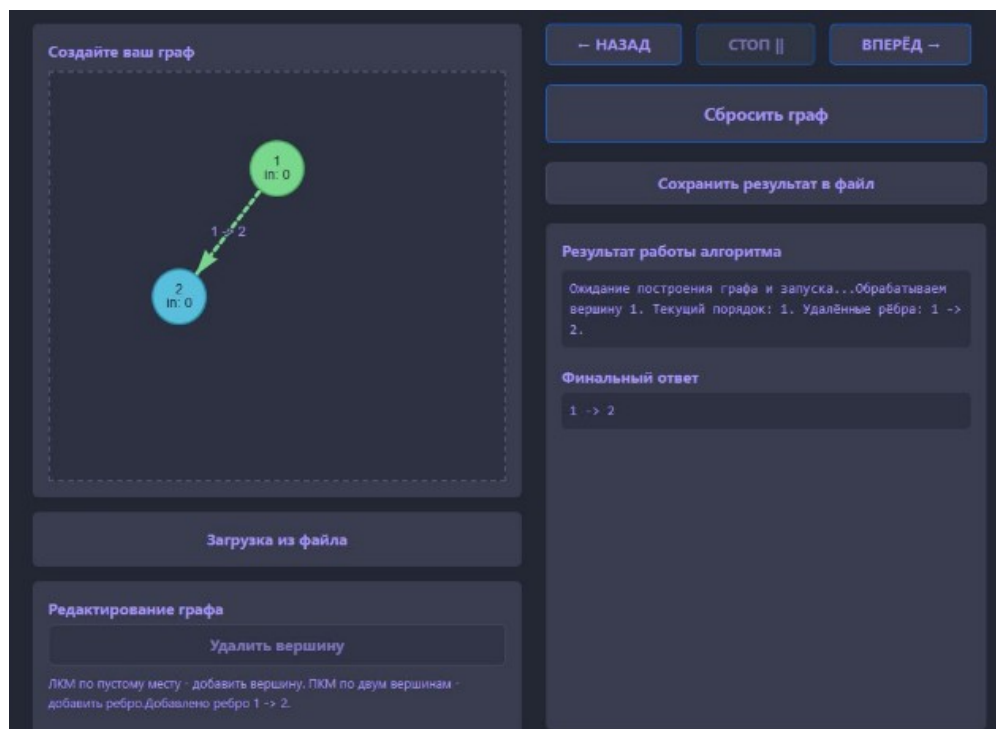


Рисунок 3 – Пример успешной работы алгоритма

И пример успешно отловленной ошибки, а именно циклического графа (рис. 4).

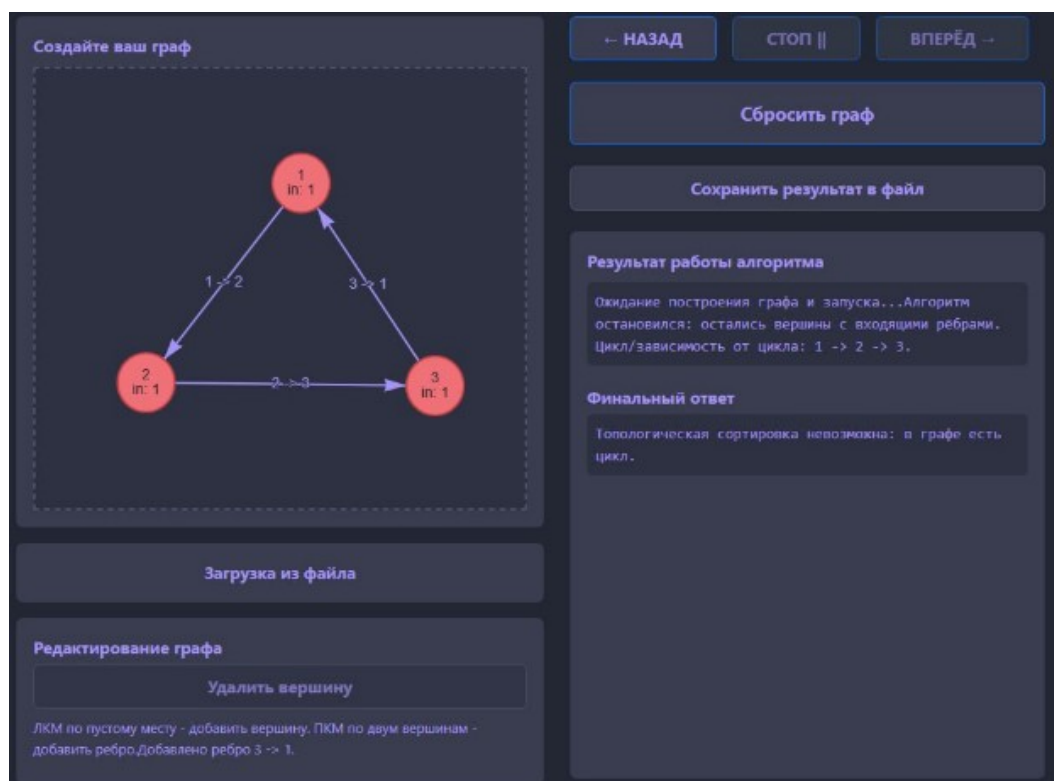


Рисунок 4 – Пример успешной обработки ошибки

2.3.3.2. Тестирование алгоритма

Тестирование проводилось на этапе 2, так как последующие этапы не подразумевают изменений в коде, отвечающем за алгоритмическую часть. В связи с малым количеством возможно проблемных ситуаций, тестирование проводилось вручную, путём модификации передаваемого json файла. Были протестированы следующие ситуации: Граф связан и ациклический, граф имеет цикл, следовательно не может быть отсортирован, граф имеет более 1 компоненты связности, граф имеет вершины, но не имеет граней. Так же были протестированы промежуточные варианты, ошибки в структуре json файла и крайние ситуации. В ходе тестирования ошибок выявлено не было. Программа отработала штатно, в случае неподходящих или некорректных входных данных была выведена информация об этом. Непредвиденного

поведения выявлено не было, что говорит о высокой надёжности программы и налаженном механизме обнаружения неисправностей.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе работы были изучены и использованы такие языки программирования, как Kotlin и JavaScript. Основная логическая составляющая была написана на языке Kotlin, а JavaScript использовался как вспомогательный при работе с графическим интерфейсом.

Так же были использованы такие библиотеки и фреймворки, как фреймворк KVision (интерфейс пользователя), библиотека vis.js Network (интерактивная визуализация графа через Kotlin/JS биндинги).
Бэкенд: Kotlin, асинхронный веб-фреймворк Ktor (порт 8080).
Сериализация данных: библиотека kotlinx.serialization (стандарт JSON).
Асинхронность: Kotlin Coroutines (для предотвращения блокировки UI-потока браузера во время сетевых запросов).

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Отзыв руководителя (рис. 5).

Отзыв
о прохождении учебной практики

Садыков Семён Русланович, студент группы 4342, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил учебную практику по теме “Новые языки программирования” с 06.07.2026 по 18.07.2026.

В течение практики обучающийся Садыков Семён Русланович выполнял задание на создание визуализатора топологической сортировки.

В результате практики обучающийся создал интерактивное веб-приложение, позволяющее выполнять сортировку как на файлах с готовыми параметрами, так и созданными вручную с помощью интерфейса. Получаемую работу обучающийся Садыков Семён Русланович выполнял активно, проявляя интерес к языку программирования Kotlin и написанию на нём бэкенд части приложения.

По результатам работы Садыкова Семёна Руслановича учебную практику оцениваю на Отлично.

Шестопалов Р.П.
Ассистент каф. МОЭВМ 15.07.2026

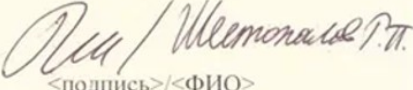

<подпись>/<ФИО>

Рисунок 5 – Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была выполнена командная разработка программного продукта — визуализатора алгоритма топологической сортировки Кана на языке Kotlin. Полностью решены все поставленные задачи: пройден вводный курс по языку Kotlin [1], составлена спецификация программы, согласован план работ и распределены роли в бригаде, освоены фреймворк KVision [2] и библиотека vis.js, что позволило создать интерактивный веб-интерфейс, реализована архитектура приложения[3]: классы Graph, Vertex, Edge, парсер JSON-файлов, алгоритм Кана с пошаговым режимом и детектированием циклов, разработан графический интерфейс с областью рисования графа, панелью управления и загрузкой данных, проведено тестирование алгоритмической части на различных типах графов (ациклические, циклические, несвязные), а также ручное тестирование интерфейса; ошибок и непредвиденного поведения не выявлено.

Созданное приложение полностью удовлетворяет исходным требованиям и может использоваться в учебном процессе для демонстрации работы алгоритма топологической сортировки. Полученные навыки программирования на Kotlin, проектирования веб-приложений и командной разработки будут полезны в дальнейшей профессиональной деятельности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Введение в Kotlin JVM [Электронный ресурс]. URL: <https://stepik.org/course/5448/syllabus> (дата обращения: 06.07.2026).
2. KVision [Электронный ресурс]. URL: <https://kvision.io/> (дата обращения: 10.07.2026).
3. Репозиторий: https://github.com/yankupchinskiy/summer_prac_TopSort.git

ПРИЛОЖЕНИЕ А

Представляется финальный вид пользовательского интерфейса приложения (рис. 6).

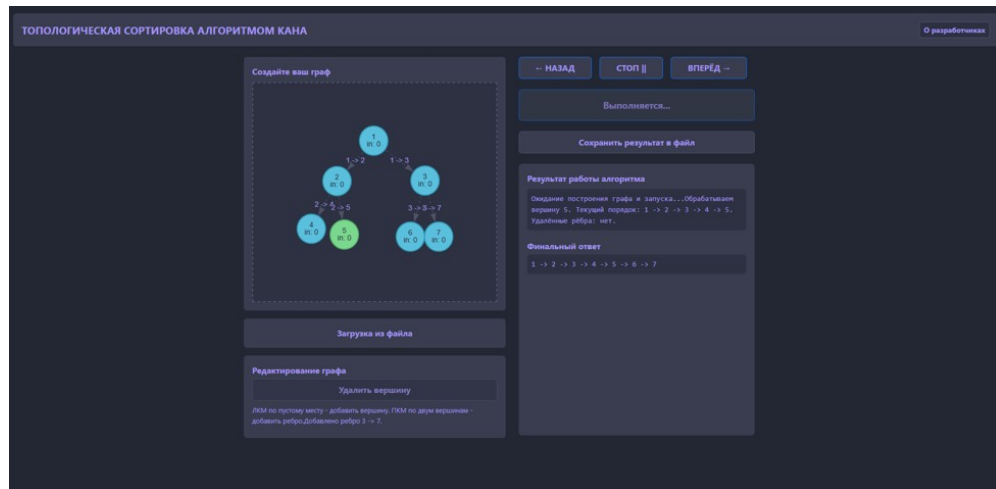


Рисунок 6 – релизный вид продукта